

Progetto 2 — SOSI Shell

Relazione tecnica

Implementazione incrementale di una shell Unix-like in C

Corso:	Sistemi Operativi e Sicurezza Informatica
Docente:	Prof. Francesco Fontanella
Progetto:	SOSI Shell
Anno accademico:	2025/2026
Studente:	Andrea Molle 78101

Indice

Parte I — Fondamenti teorici e concettuali

1. La shell e il suo ciclo di vita
2. Processi e system call: fork, execv, waitpid
3. Built-in e comandi esterni: perché cd va gestito dalla shell
4. Variabili d'ambiente e ricerca in PATH
5. File descriptor, dup2 e redirezione dell'output
6. La pipe
7. L'array circolare per la cronologia

Parte II — Descrizione dell'implementazione

8. Architettura generale e flusso di esecuzione
9. Scelte implementative
10. Gestione degli errori
11. Testing

Parte III — Diario dell'interazione con l'IA

Limiti ed estensioni

Parte I — Fondamenti teorici e concettuali

Per ciascun argomento seguo l'approccio *Cos'è / Come funziona / Perché nel mio progetto*.

1. La shell e il suo ciclo di vita

Cos'è

Una shell è un programma in spazio utente che fa da intermediario tra l'utente e il kernel del sistema operativo. Non è parte del kernel: è un normale processo che, leggendo comandi testuali, traduce le intenzioni dell'utente in chiamate alle primitive di sistema (*system call*). È quindi un interprete di comandi.

Come funziona

Una shell interattiva implementa un ciclo (il cosiddetto *REPL*, *Read-Eval-Print Loop*) che si ripete fino alla terminazione:

1. prompt — stampa di una stringa che indica che la shell attende input;
2. read — lettura di una riga da standard input;
3. parse — suddivisione della riga in token e riconoscimento dei metacaratteri (`>`, `&`, `|`);
4. dispatch — distinzione tra comando built-in e comando esterno;
5. execute — esecuzione diretta (built-in) o tramite processo figlio (esterno);
6. ritorno al passo 1.

Perché nel mio progetto

Questo ciclo è il cuore del `main()`: il `while (TRUE)` richiama in sequenza `prompt()`, `read_command()`, `built_ins()` e, per i comandi esterni, la coppia `find_path() + fork/execv`. Ho mantenuto la logica distribuita nelle funzioni dello skeleton (e non concentrata nel `main`) proprio perché ogni passo del REPL corrisponde a una responsabilità ben distinta.

2. Processi e system call: fork, execv, waitpid

Cos'è

Un *processo* è l'istanza in esecuzione di un programma, dotata di un proprio spazio di indirizzamento, descrittori di file e contesto. Le *system call* sono il punto di ingresso controllato dallo spazio utente verso i servizi del kernel: per creare ed eseguire programmi esterni la shell usa `fork()`, `execv()` e `waitpid()`.

Come funziona

Una system call non è una semplice chiamata di funzione: provoca una *transizione dallo user mode al kernel mode*. Il processo carica il numero della chiamata e i parametri in registri prestabiliti ed esegue un'istruzione di trap (es. `syscall` su x86-64); la CPU passa in modalità privilegiata, il kernel esegue il servizio richiesto e al ritorno si torna in user mode con il valore di ritorno. Le tre primitive hanno una semantica precisa:

- `fork()` duplica il processo chiamante creando un figlio quasi identico; restituisce due volte: 0 nel figlio e il PID del figlio nel padre. Da questo valore i due processi capiscono chi sono.
- `execv(path, argv)` sostituisce l'immagine del processo corrente con quella del programma indicato da `path`; se ha successo non ritorna, perché il codice chiamante è stato rimpiazzato. Ritorna solo in caso di errore.
- `waitpid(pid, &status, 0)` sospende il padre finché il figlio indicato non termina, raccogliendone lo stato di uscita ed evitando che resti uno zombie.

Il pattern canonico è *fork-then-exec*: si duplica il processo e solo il figlio chiama `execv`, così la shell (il padre) sopravvive all'esecuzione del comando.

Perché nel mio progetto

Senza `fork` la shell non potrebbe eseguire un programma esterno senza suicidarsi (perché `execv` rimpiazza l'immagine). Creando un figlio, è il figlio a essere sostituito dal comando, mentre il padre continua il REPL. `waitpid` mi serve per implementare il foreground: in quel caso il padre attende la fine del figlio prima di ristampare il prompt.

```
pid = fork();
if (pid < 0) { perror("fork"); continue; }
if (pid != 0) { /* PADRE */
    if (w) waitpid(pid, &s, 0); /* attende solo in foreground */
} else { /* FIGLIO */
    if (outRe) redirStdout(outName);
    execv(path, pars);
    perror("execv"); /* raggiunto solo se execv fallisce */
    exit(1);
}
```

Listing 1 — Pattern fork/execv/waitpid nel main

Un'alternativa sarebbe `system()`, che però invoca a sua volta `/bin/sh` e non permette di controllare foreground/background, redirezione e pipe a basso livello: per gli obiettivi didattici del progetto sarebbe stata una scorciatoia che nasconde proprio i meccanismi da studiare.

3. Built-in e comandi esterni: perché cd va gestito dalla shell

Cos'è

Un comando *built-in* è eseguito direttamente dal processo shell; un comando *esterno* è un eseguibile separato lanciato in un processo figlio. La distinzione non è arbitraria: alcuni comandi devono modificare lo stato interno della shell stessa.

Come funziona

La *working directory* è un attributo del processo, modificabile con la system call `chdir()`. Ogni processo ha la propria copia: un figlio che chiama `chdir` cambia solo la sua directory corrente, non quella del padre.

Perché nel mio progetto

Se implementassi `cd` come comando esterno, la shell `forkerebbe` un figlio, il figlio chiamerebbe `chdir` e poi terminerebbe: il cambiamento morirebbe con lui e la shell interattiva resterebbe nella directory di partenza. Per questo `cd` (insieme a `exit`, `quit`, `history`) è gestito dentro `built_ins()`, che chiama `chdir` nel processo shell. È la stessa ragione per cui `exit` deve terminare la shell e non un figlio.

4. Variabili d'ambiente e ricerca in PATH

Cos'è

L'*ambiente* è un insieme di coppie `NOME=valore` che il kernel passa a ogni processo all'avvio e che i figli ereditano. Variabili come `USER`, `HOME` e `PATH` descrivono il contesto d'esecuzione. `PATH` è l'elenco, separato da `:`, delle directory in cui cercare gli eseguibili.

Come funziona

`getenv("PATH")` restituisce la stringa della variabile. La shell la scandisce directory per directory: per ogni directory costruisce il path candidato `directory/comando` e verifica con `stat()` se esiste un file regolare eseguibile. La prima corrispondenza vince (ordine di precedenza).

Perché nel mio progetto

`find_path()` realizza esattamente questa scansione usando `strtok_r()` anziché `strtok()`: `strtok` mantiene uno stato statico interno e non è rientrante, mentre `strtok_r` riceve un puntatore di salvataggio esplicito (`saveptr`). Ho scelto la versione rientrante per evitare interferenze con altri usi di `strtok` (in particolare la tokenizzazione della riga in `read_command`). La funzione gestisce inoltre il caso del

path assoluto: se il comando inizia con / la scansione di PATH viene saltata e si verifica direttamente il file indicato.

```
env_path = getenv("PATH");
strncpy(local_path, env_path, MAX_STRING - 1);
token = strtok_r(local_path, ":", &saveptr);
while (token != NULL) {
    concat(token, com, path);          /* token + "/" + com */
    if (check_file(path, TRUE)) return TRUE;
    token = strtok_r(NULL, ":", &saveptr);
}
```

Listing 2 — Scansione di PATH con strtok_r

Per HOSTNAME ho fatto una scelta diversa da quella suggerita dalla traccia: la variabile non è quasi mai esportata nell'ambiente (è una variabile di shell, non d'ambiente), quindi un `getenv("HOSTNAME")` restituirebbe spesso NULL. Ho quindi usato la system call `gethostname()`, che legge il nome dal kernel ed è affidabile.

5. File descriptor, dup2 e redirezione dell'output

Cos'è

Un *file descriptor* (fd) è un intero che indicizza la tabella dei file aperti del processo. Per convenzione 0 è stdin, 1 è stdout, 2 è stderr. La *redirezione* consiste nel far puntare uno di questi fd a un file invece che al terminale.

Come funziona

`open()` restituisce il primo fd libero associandolo al file. `dup2(oldfd, newfd)` chiude `newfd` (se aperto) e lo fa diventare un duplicato di `oldfd`: da quel momento le due voci puntano alla stessa risorsa. Ridirigere stdout su un file significa quindi `dup2(fd_file, STDOUT_FILENO)`.

Perché nel mio progetto

La redirezione va eseguita *nel figlio*, dopo il `fork` e prima di `execv`: così riguarda solo il processo che eseguirà il comando e non altera lo stdout della shell. In `redirStdout()` apro il file con `O_WRONLY | O_CREAT | O_TRUNC`, redirigo sia stdout sia stderr (per rendere osservabili eventuali errori del comando, come suggerito dalla traccia) e chiudo il fd originale, ormai superfluo perché duplicato.

```
int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd < 0) { perror("open"); exit(1); }
dup2(fd, STDOUT_FILENO);
dup2(fd, STDERR_FILENO);
close(fd);
```

Listing 3 — Redirezione di stdout e stderr

6. La pipe

Cos'è

Una *pipe* è un canale di comunicazione unidirezionale tra processi: un buffer gestito dal kernel con due estremità, una di lettura e una di scrittura. È il meccanismo che realizza il costrutto `cmd1 | cmd2`, collegando lo `stdout` di `cmd1` allo `stdin` di `cmd2`.

Come funziona

`pipe(fd)` crea il canale e popola l'array `fd`: `fd[0]` è il lato di lettura, `fd[1]` quello di scrittura. Poiché i `fd` vengono ereditati attraverso `fork`, due processi figli possono condividere la stessa *pipe*. Il primo figlio reindirizza il proprio `stdout` su `fd[1]`; il secondo reindirizza il proprio `stdin` su `fd[0]`. Punto cruciale: *tutti i processi devono chiudere i lati della pipe che non usano*. Se anche un solo processo lascia aperto il lato di scrittura, il lettore non riceve mai l'EOF e si blocca in attesa indefinita (deadlock).

Perché nel mio progetto

La Fase 3 è gestita da `run_pipeline()`, che crea la *pipe*, esegue due `fork` e imposta le redirezioni con `dup2`. Il padre, dopo i `fork`, chiude entrambi i descrittori: è la chiusura del lato di scrittura nel padre che permette al secondo comando di vedere l'EOF quando il primo termina. Ho verificato l'assenza di deadlock con un test di stress (50 pipeline consecutive).

```
pipe(fd);
if ((pid1 = fork()) == 0) {           /* figlio 1: scrive */
    dup2(fd[1], STDOUT_FILENO);
    close(fd[0]); close(fd[1]);
    execv(path1, p1); perror("execv"); exit(1);
}
if ((pid2 = fork()) == 0) {           /* figlio 2: legge */
    dup2(fd[0], STDIN_FILENO);
    close(fd[1]); close(fd[0]);
    execv(path2, p2); perror("execv"); exit(1);
}
close(fd[0]); close(fd[1]);           /* padre: essenziale per l'EOF */
waitpid(pid1, &s, 0); waitpid(pid2, &s, 0);
```

Listing 4 — Nucleo della pipeline (semplificato)

7. L'array circolare per la cronologia

Cos'è

La *history* conserva gli ultimi $N = 10$ comandi. La struttura dati scelta è un *buffer circolare*: un array di dimensione fissa in cui l'indice di scrittura, una volta raggiunta la fine, riparte da capo sovrascrivendo gli elementi più vecchi.

Come funziona

Due indici governano la struttura: `h_i`, posizione della prossima cella da scrivere (avanza in modulo N), e `h_n`, contatore progressivo assoluto dei comandi. `h_n` non viene mai azzerato: garantisce che la numerazione mostrata all'utente resti

coerente anche dopo molte sovrascritture (es. i comandi 7–16 dopo averne digitati 16). L'avanzamento circolare è $h_i = (h_i + 1) \% N$ e la scansione all'indietro (dal più recente) $idx = (idx - 1 + N) \% N$.

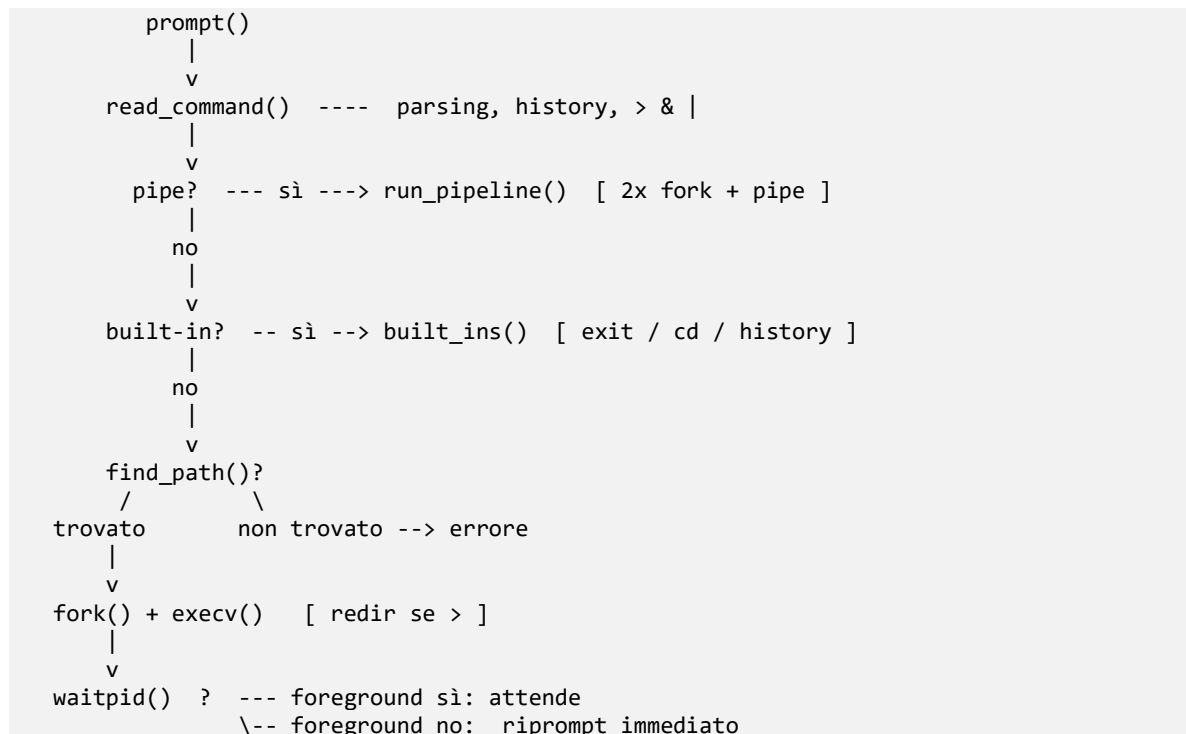
Perché nel mio progetto

Un buffer circolare offre inserimento e accesso in tempo costante senza spostamenti di memoria, a differenza di una coda realizzata con un array a scorrimento (che richiederebbe di traslare tutti gli elementi a ogni inserimento) o di una lista collegata (che introdurrebbe allocazioni dinamiche e overhead di puntatori per una dimensione comunque fissa). Per 10 elementi è la scelta più semplice ed efficiente. Ho separato il numero assoluto n dalla posizione fisica proprio per disaccoppiare cosa vede l'utente da dove è memorizzato.

Parte II — Descrizione dell'implementazione

8. Architettura generale e flusso di esecuzione

Il programma è un singolo sorgente `SOSI_shell.c` organizzato in funzioni con responsabilità distinte. Il `main` contiene solo il ciclo REPL e il pattern `fork/exec`; tutta la logica specifica è delegata alle funzioni. Lo schema seguente riassume il flusso di una riga di comando.



Le funzioni principali e il loro ruolo:

- `prompt()` — compone e stampa `user@host:cwd$`;
- `read_command()` — legge la riga, gestisce la history, tokenizza e riconosce `>`, `&`, `|`;
- `built_ins()` — riconosce ed esegue i built-in;
- `find_path()` / `check_file()` / `concat()` — risoluzione e verifica dell'eseguibile;
- `redirStdout()` — redirectione dell'output;
- `run_pipeline()` — esecuzione coordinata di due comandi con pipe;
- `manage_history()`, `get_history()`, `update_history()`, `show_history()` — gestione della cronologia.

9. Scelte implementative

Fase 0 — pulizia dello skeleton

Lo skeleton non compilava. Ho corretto: **(i)** la variabile di ciclo `i` usata ma non dichiarata nel `main`; **(ii)** la firma di `built_ins`, disallineata tra prototipo (`char *p[]`) e definizione (`char *c, char *p[]`): ho adottato la versione a un solo argomento, coerente con l'unica chiamata presente; **(iii)** il prototipo di `check_file`, privo del parametro `exe` presente nella definizione. Ho inoltre rimosso le macro `inc` e `dec` dello skeleton: `#define inc(val) val++ % H_SIZE`, usata come `h_i = inc(h_i)`, espande in `h_i = h_i++ % H_SIZE`, che modifica `h_i` due volte senza punto di sequenza — *comportamento indefinito*. L'ho sostituita con aritmetica modulare esplicita.

Parsing unico con macchina a stati

Ho scelto di gestire `>`, `&` e `|` in un'unica scansione di token in `read_command`, usando un puntatore `cur` all'array di destinazione corrente. Quando incontro `|`, chiudo il primo array con `NULL` e faccio puntare `cur` al secondo array (`pars2`). Questo evita una seconda passata sulla riga e mantiene il codice lineare. I metacaratteri non vengono mai copiati in `pars`, così `execv` riceve solo gli argomenti reali e sempre un array terminato da `NULL` (requisito di `execv`).

History prima della tokenizzazione

`manage_history` è invocata prima di `strtok`, perché `strtok` altera la stringa inserendo `\0` al posto degli spazi: registrare il comando dopo significherebbe salvarne solo il primo token. Per il recupero (`!!`, `!N`, `!str`) riscrivo direttamente il buffer di input con il comando recuperato, così la successiva tokenizzazione opera in modo trasparente sul comando espanso.

getcwd invece di \$PWD

Nel prompt uso `getcwd()` anziché `getenv("PWD")` perché `$PWD` non viene aggiornata automaticamente dopo `chdir`: leggerla darebbe un valore obsoleto. Per coerenza, dopo ogni `cd` aggiorno comunque `$PWD` con `setenv`.

10. Gestione degli errori

La shell deve essere robusta: *nessun input deve provocare la terminazione anomala*. Le strategie adottate:

- controllo del valore di ritorno di tutte le system call critiche (`fork`, `execv`, `open`, `dup2`, `pipe`, `chdir`, `stat`), con messaggi tramite `perror()`;
- comando inesistente, directory inesistente, numero di history assente: messaggio d'errore e ritorno al prompt, senza uscire;
- errori di sintassi della pipe (pipe a inizio o fine riga): rilevati in `run_pipeline` con messaggi distinti per comando sinistro/destro mancante;
- uso sistematico di funzioni con limite di dimensione (`strncpy`, `strncat`) e terminazione esplicita delle stringhe, per prevenire buffer overflow;
- gestione dell'EOF in `get_string`: se `fgets` restituisce NULL (Ctrl-D o fine di un file rediretto in input) la shell termina in modo ordinato, anziché iterare su un buffer in stato indefinito — un bug latente dello skeleton originale;
- assenza di *memory leak*: gli array degli argomenti vengono liberati all'inizio di ogni nuovo comando e, alla terminazione, da un handler registrato con `atexit`. Verificato con `valgrind` (0 errors, no leaks).

11. Testing

I test sono stati eseguiti in modo non interattivo inviando script di comandi sullo `stdin` della shell e confrontando l'output con quello atteso, oltre a verifiche interattive. La tabella riporta gli scenari principali (gli ID corrispondono a quelli della traccia).

ID	Comando / azione	Esito
T0.1	<code>gcc -Wall -Wextra -g</code>	Compilazione senza warning
T0.3	Invio a vuoto	Riprompt, nessuna uscita
T1.2–1.3	<code>ls</code> , <code>ls -l /tmp</code>	Argomenti passati correttamente
T1.4	<code>cd /tmp</code> ; <code>pwd</code>	Directory cambiata in <code>/tmp</code>
T1.5	<code>cd</code> senza argomenti	Ritorno alla home
T1.6	<code>cd dir_inesistente</code>	Messaggio d'errore, shell attiva

ID	Comando / azione	Esito
T1.7–1.8	sleep 5 / sleep 5 &	Attesa / ritorno immediato al prompt
T1.9	ls -l > out.txt	File creato con l'output
T1.10	comando_inesistente	command not found, shell attiva
T1.11	exit / quit	Terminazione ordinata
T2.1	history all'avvio	Nessun crash
T2.2	date, who, cal, history	Comandi numerati in ordine inverso
T2.3	!!	Comando più recente stampato ed eseguito
T2.4	!N	Comando N eseguito se presente
T2.5	> 10 comandi, poi history	Solo i 10 più recenti, numerazione assoluta
T2.6	!999	command !999 not found
T2.8	!str (bonus)	Ultimo comando che inizia con str
T3.1–3.3	ls -l /etc grep conf, cat /etc/passwd grep root, ps aux grep bash	Output filtrato correttamente
T3.4–3.5	grep bash, ls	Errore di sintassi, nessun crash
T3.6–3.7	bogus grep x, ls bogus	Errore sul comando mancante, shell attiva
T3.8	50 pipeline ripetute	Nessun deadlock; ritorno sempre al prompt
Mem	valgrind --leak-check=full	0 errors, no leaks possible

Di seguito riporto alcuni estratti dalla sessione di testing interattivo eseguita sulla VM di sviluppo (Ubuntu 22.04, hostname kdev).

Compilazione e avvio (T0.1)

```
ubuntu@kdev:~/progetto_shell$ gcc -Wall -Wextra -g SOSI_shell.c -o sosi_shell
ubuntu@kdev:~/progetto_shell$ ./sosi_shell
ubuntu@kdev:~/progetto_shell$
```

Nessun warning, il prompt appare correttamente con utente, hostname e directory.

Built-in e comandi esterni (Fase 1)

```
ubuntu@kdev:~/progetto_shell$ ls
Makefile  SOSI_shell.c  sosi_shell
ubuntu@kdev:~/progetto_shell$ cd /tmp
ubuntu@kdev:/tmp$ cd
ubuntu@kdev:~$ cd /cartella_pippo
cd: /cartella_pippo: no such directory
```

```
ubuntu@kdev:~$ echo test_output > /tmp/test_redir.txt
ubuntu@kdev:~$ cat /tmp/test_redir.txt
test_output
ubuntu@kdev:~$ prova
ERROR: command not found: prova
```

Si nota il prompt che si aggiorna dopo `cd`, la gestione dell'errore per una directory inesistente, la redirectione dell'output su file verificata con `cat`, e il messaggio per un comando non trovato.

History con overflow circolare (Fase 2)

Dopo aver eseguito più di 10 comandi (tra cui `echo` a ... `echo` 1):

```
ubuntu@kdev:~$ history
29 history
28 echo 1
27 echo k
26 echo j
25 echo i
24 echo h
23 echo g
22 echo f
21 echo e
20 echo d
```

La numerazione è assoluta (20–29) e vengono mostrati solo gli ultimi 10 comandi, confermando il corretto funzionamento del buffer circolare. Il recupero con `!echo` ha rieseguito con successo l'ultimo comando che iniziava con `echo`, mentre `!999` ha prodotto il messaggio `command !999 not found`.

Pipeline ed errori di sintassi (Fase 3)

```
ubuntu@kdev:~$ cat /etc/passwd | head -5
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
ubuntu@kdev:~$ echo ciao mondo | wc -w
2
ubuntu@kdev:~$ | ls
ERROR: syntax error near '|' (missing left command)
ubuntu@kdev:~$ ls |
ERROR: syntax error near '|' (missing right command)
```

Le pipeline producono l'output corretto; gli errori di sintassi vengono rilevati con messaggi distinti senza crash.

Valgrind (assenza di memory leak)

```
ubuntu@kdev:~/progetto_shell$ valgrind --leak-check=full ./sosi_shell
==5023== Memcheck, a memory error detector
==5023== Command: ./sosi_shell
[...sessione interattiva con ls, cd, echo, pipe, history, exit...]
==5023== HEAP SUMMARY:
==5023==    in use at exit: 0 bytes in 0 blocks
==5023==   total heap usage: 20 allocs, 20 frees, 16,458 bytes allocated
==5023== All heap blocks were freed -- no leaks are possible
==5023== ERROR SUMMARY: 0 errors from 0 contexts
```

Valgrind conferma 20 allocazioni e 20 liberazioni, con zero errori e zero leak.

Un dettaglio emerso nei test automatici: quando l'input è una pipe (non un terminale), lo stdout della shell è *fully buffered*, quindi l'eco di un comando recuperato dalla history poteva comparire dopo l'output del comando stesso. Ho aggiunto un `fflush(stdout)` dopo l'eco per garantire l'ordine corretto.

Parte III — Diario dell'interazione con l'IA

Strumenti utilizzati

Ho usato **Claude** (Anthropic, famiglia Opus 4.x) come assistente, in modalità conversazionale, fornendo lo skeleton e le specifiche del progetto. L'IA è stata usata per: generare l'ossatura del codice fase per fase, spiegare alcuni meccanismi (in particolare la chiusura dei descrittori nelle pipe), supportare il debugging, preparare la relazione in LaTeX e generare un piano di test strutturato per la verifica sulla VM. La compilazione, l'esecuzione dei test e la verifica con valgrind sono state svolte interamente da me sulla VM di sviluppo (kdev, Ubuntu 22.04).

Cronologia delle interazioni

Di seguito le schede delle fasi più significative.

Scheda — Fase 0/1 (skeleton e shell di base)

Strumento IA	Claude Opus 4.x
Obiettivo	Far compilare lo skeleton e implementare <code>prompt</code> , <code>parsing</code> , <code>built-in</code> , <code>find_path</code> , <code>fork/execv/waitpid</code> , & <code>e ></code>
Prompt principale	«Analizza lo skeleton e le specifiche; correggi gli errori di compilazione e implementa la Fase 1 mantenendo la struttura modulare.»
Risposta dell'IA	Codice completo e compilabile al primo tentativo
Funzionava?	Sì, ma ho dovuto verificare di persona ogni correzione di Fase 0
Modifiche apportate	Verificata la scelta su <code>built_ins</code> a un argomento; preferito <code>gethostname()</code> a <code>getenv("HOSTNAME")</code> dopo aver constatato che la variabile non era esportata sul mio sistema
Iterazioni	1
Cosa ho imparato	La differenza tra variabili di shell e variabili d'ambiente; perché il prompt deve usare <code>getcwd</code> e non <code>\$PWD</code>

Scheda — Fase 2 (history)

Strumento IA	Claude Opus 4.x
Obiettivo	Implementare l'array circolare, history, !!, !N, bonus !str
Prompt principale	«Implementa la history su array circolare con numerazione assoluta e recupero dei comandi.»
Funzionava?	Parzialmente: l'ordine di stampa dell'eco del comando recuperato era sbagliato nei test con input rediretto
Modifiche apportate	Aggiunto fflush(stdout) dopo l'eco; capito che il problema era il buffering di stdout (line-buffered su tty, fully-buffered su pipe) e non la logica
Iterazioni	2
Cosa ho imparato	Il comportamento del buffering della libreria standard a seconda che lo standard output sia un terminale o un file/pipe

Scheda — Fase 3 (pipe)

Strumento IA	Claude Opus 4.x
Obiettivo	Pipeline tra due comandi senza deadlock e con gestione degli errori di sintassi
Funzionava?	Quasi: il caso grep bash (pipe iniziale) veniva scambiato per riga vuota e saltato silenziosamente
Modifiche apportate	Spostato il controllo pipeRe prima del controllo riga vuota / built-in nel main, così la pipe iniziale viene riconosciuta come errore di sintassi
Iterazioni	2
Cosa ho imparato	L'importanza di chiudere il lato di scrittura della pipe nel padre: senza, il secondo comando non riceve l'EOF e la shell va in deadlock

Scheda — Testing e verifica sulla VM

Strumento IA	Claude Opus 4.x
Obiettivo	Ottenere un piano di test strutturato, fase per fase, da eseguire sulla VM per verificare il funzionamento della shell prima della consegna
Prompt principale	«Fammi testare il progetto sulla VM: dammi i comandi da scrivere nel terminale e dimmi quale output aspettarmi.»
Risposta dell'IA	Piano di test in 10 step: compilazione, comandi Fase 1 (built-in, redirezione, background, errori), Fase 2 (history, overflow circolare, recupero !!/!N/!str), Fase 3 (pipeline, errori di sintassi), valgrind, input rediretto da file, Ctrl-D

Funzionava?	Sì: tutti i test sono passati senza bug. L'IA ha anche saputo spiegare perché !1 risultava <code>not found</code> (il comando 1 era già uscito dal buffer circolare a quel punto)
Modifiche apportate	Nessuna modifica al codice; ho inserito nella relazione gli estratti dell'output reale dalla VM come evidenza del testing
Iterazioni	1
Cosa ho imparato	L'importanza di avere un piano di test sistematico: seguendo gli step uno per uno ho verificato ogni funzionalità in isolamento, cosa che non avrei fatto con lo stesso rigore testando a mano libera

Analisi critica del codice generato

La prima versione adottava una strategia *alloca una volta e riusa* per le celle degli argomenti. Apparentemente efficiente, conteneva però un memory leak reale: quando un comando successivo era più corto del precedente, il terminatore NULL sovrascriveva un puntatore già allocato, perdendone il riferimento. Il bug non era visibile a occhio né dai test funzionali: l'ho scoperto solo eseguendo `valgrind`, che segnalava `3.072 bytes definitely lost`. La correzione (liberare gli array all'inizio di ogni comando + handler `atexit`) ha portato a `0 errors, no leaks`. È l'esempio più chiaro del fatto che *codice che compila e funziona non equivale a codice corretto*: senza capire la gestione della memoria non avrei nemmeno saputo cosa cercare.

Errori e problemi incontrati

I tre problemi sopra (buffering, pipe iniziale, memory leak) non sono stati risolti dall'IA in autonomia: sono emersi solo con test ed analisi mirate da parte mia, e in due casi su tre la diagnosi (buffering, ordine dei controlli nel `main`) è stata mia. L'IA è stata utile per scrivere rapidamente la correzione una volta individuata la causa.

Bilancio finale

L'uso dell'IA ha accelerato nettamente la stesura del codice meccanico (tokenizzazione, gestione degli array), lasciandomi più tempo per ragionare sui meccanismi del sistema operativo. Tuttavia *il valore didattico è venuto dalle fasi di verifica*: i bug più istruttivi (memory leak, deadlock, buffering) li ho compresi a fondo solo testando e analizzando, non leggendo la prima risposta. Anche il piano di test generato dall'IA si è rivelato utile: mi ha dato una traccia sistematica da seguire sulla VM, permettendomi di coprire tutti gli scenari della traccia senza dimenticarne nessuno e di raccogliere l'output reale da inserire nella relazione. Se rifacessi il progetto, userei l'IA allo stesso modo per l'ossatura e il testing, ma anticiperei l'uso di `valgrind` e dei test su input rediretto, perché è lì che si impara davvero *under the hood*.

Limiti ed estensioni

Limiti

Una sola pipe per riga; nessuna gestione delle virgolette per argomenti con spazi; nessun globbing; redirectione dell'input (<) e append (>>) non implementati; i processi in background non vengono raccolti con un handler SIGCHLD, quindi possono restare zombie fino alla terminazione della shell.

Estensioni realizzate

Recupero !str (bonus); redirectione di stderr insieme a stdout; abbreviazione della home con ~ nel prompt; gestione dell'EOF; assenza di memory leak verificata con valgrind.

Estensioni possibili

Pipeline multiple (generalizzando run_pipeline a n comandi con un ciclo di pipe/fork); raccolta dei processi background con waitpid(WNOHANG) o handler SIGCHLD; comando jobs; supporto a < e >>.